King Fahd University of Petroleum and Minerals
College of Computing and Mathematics

Information & Computer Science Department



ICS 202
Data Structures and Algorithms

Unit 04
Stacks and Queues

Term 261

Reading Assignment

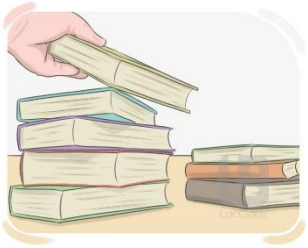
- "Data Structures and Algorithms in Java", 4th Edition, Adam Drozdek, Cengage Learning, ISBN 978-9814239233
  - Chapter 4
  - Self Reading:
    - Section 4.1 ArrayList and LinkedList

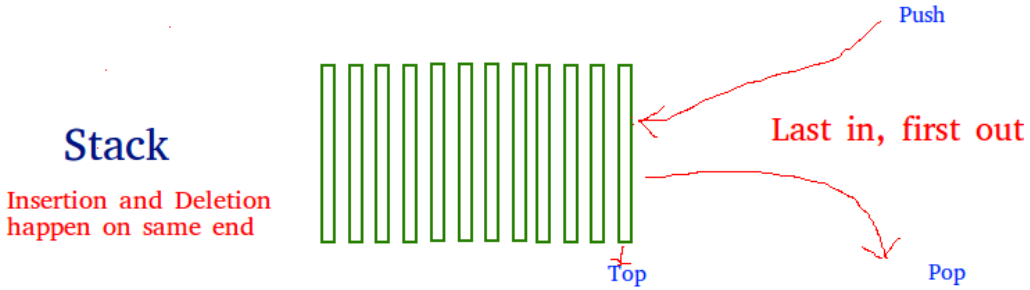
Objectives

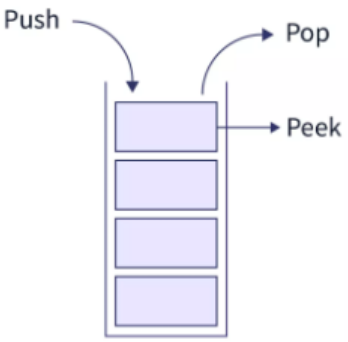
- Discuss the following topics:
  - Stacks
    - What is a Stack?
    - Stack implementation using array.
    - Stack implementation using linked list.
    - Applications of Stacks
  - Queues
  - Priority Queues
  - Case Study: Exiting a Maze [Self-Reading]

Stacks

- A stack is a linear data structure in which elements are ACCESSED (inserted and removed) from the same end, called the top, following the LIFO (Last In, First Out) principle.
  - To add (push) an item to the stack, it must be placed on the top of the stack.
  - To remove (pop) an item from the stack, it must be removed from the top of the stack too.

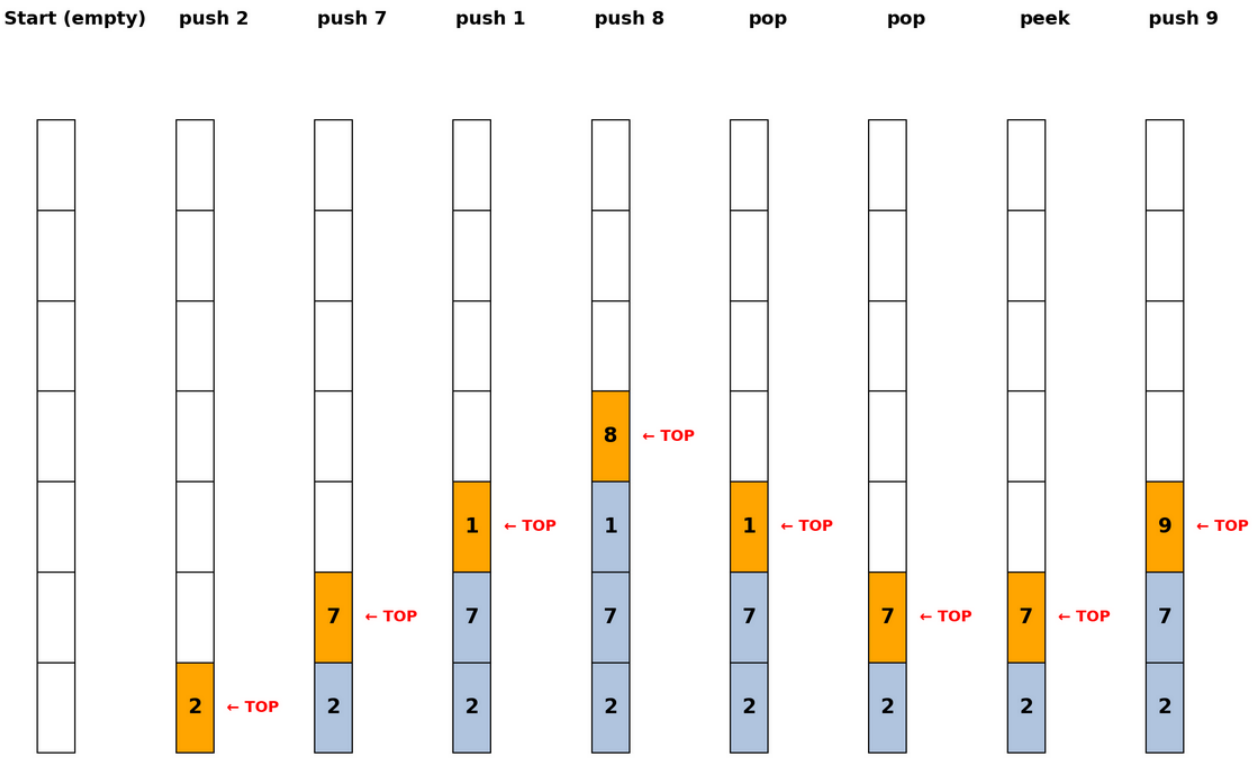






Stack: Example

- push(2), push(7), push(1), push(8), pop, pop, peek, push(9)



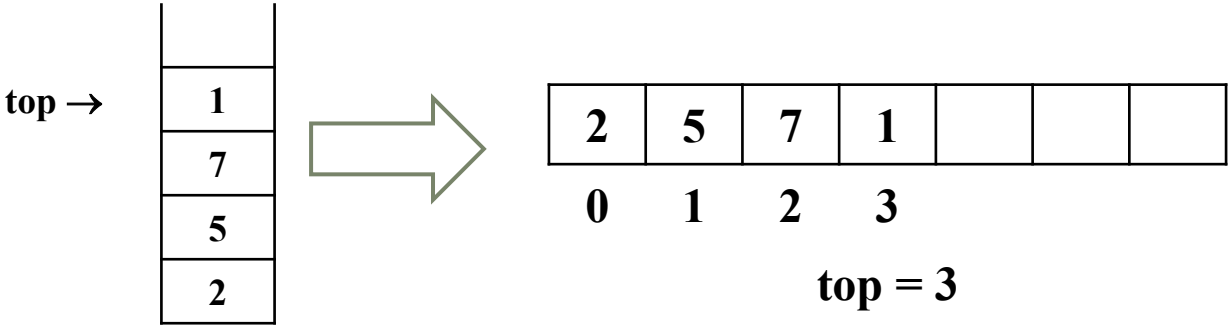
1. **Start:** `[]` (empty stack)
2. **push 2:** `[2]`
3. **push 7:** `[2, 7]`
4. **push 1:** `[2, 7, 1]`
5. **push 8:** `[2, 7, 1, 8]`
6. **pop:** removes top (8) → `[2, 7, 1]`
7. **pop:** removes top (1) → `[2, 7]`
8. **peek:** shows top → **7**
   (stack unchanged: `[2, 7]`)
9. **push 9:** `[2, 7, 9]`

✅ **Final stack:** `[2, 7, 9]`

Top element = **9**

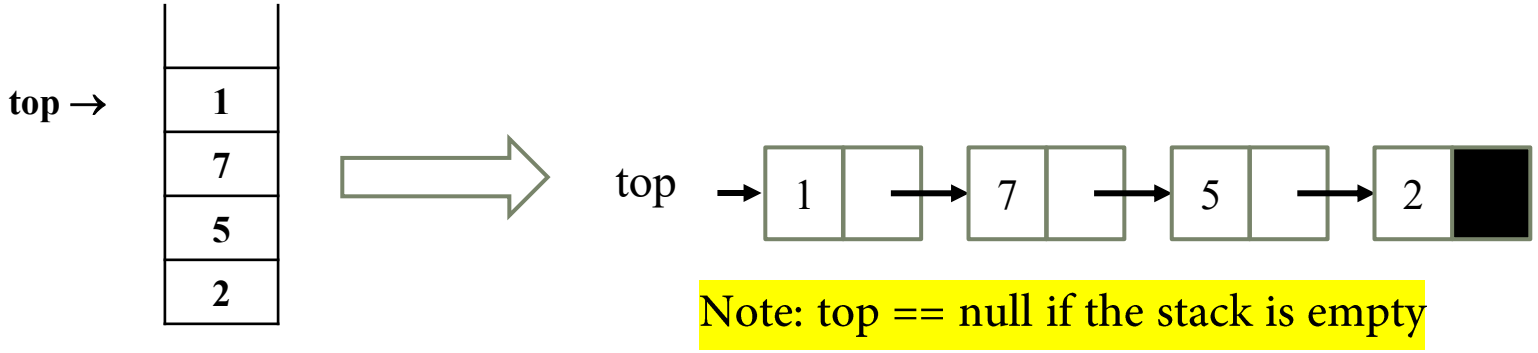
Stack Implementations

- We shall study two stack implementations:
  - StackAsArray
    - The underlying data structure is A GENRIC array of type T[ ].



Note: top == -1 if the stack is empty

  - StackAsLinkedList
    - The underlying data structure is a LinkedList.



Note: top == null if the stack is empty

Stack

- State:
  - List/array
- Behavior
  - push(item)
  - pop()
  - peek()
  - isEmpty()
  - isFull()

StackAsArray Implementation

- In the StackAsArray implementation that follows,
  - the top of the stack is array[top]
  - the bottom is array[0]
  - Where top +1 is represents the number of actual elements in the stack.

- The constructor's single parameter, capacity, specifies the maximum number of items that can be stored in the stack.
  - The variable array is initialized to be an array of length capacity

```
public class StackAsArray<T> {

    private T[] array;
    private final int capacity;
    private int top;

    public StackAsArray(int capacity) {
        this.capacity = capacity;
        this.top = -1;
        this.array = (T[]) new Object[capacity];
    }
```

```
    public boolean isEmpty() {
        return this.top == -1;
    }

    public boolean isFull() {
        return this.top + 1 == this.capacity;
    }

    public T peek() {
        if (isEmpty())
            return null;
        else
            return this.array[this.top];
    }
```

StackAsArray…

- For the push operation:
  - Check whether the stack is full
    - If full, report stack overflow
  - Otherwise: increment top
  - Insert the new element at position top.

```
public boolean push(T e)  {
    if (isFull())
        return false;
    this.array[++this.top] = e;
    return true;
}
```

- For the pop operation:
  - Check whether the stack is empty.
    - If empty, report stack underflow.
  - Otherwise, retrieve the element at position top.
  - Decrement top.
  - Return the removed element.

```
public T pop() {
    if(this.isEmpty()){
        return null;
    }
    T result = this.array[this.top];
    //this.array[this.top] = null;
    this.top--;
    return result;
}
```

```
public String toString() {
    String str = "";
    for(int i = 0; i<= this.top; i++){
        str = str + this.array[i] + " ";
    }
    return str;
}
```

push method returns true/false

True: successful operations

False: unsuccessful operation

StackAsLinkedList Implementation

```
public class StackAsList<T> {

        private class Node {
        T info;
        Node next;

        public Node(T e){
            info = e;
            next = null;
        }
    }

    private Node top = null;

    public boolean isEmpty(){
        return top == null;
    }

    public void push (T e) {
        Node tmp = new Node(e);
        tmp.next = top;
        top= tmp;
    }

    public T pop () {
        if(isEmpty())
            return null;
        T e = top.info;
        top = top.next;
        return e;
    }
```

```
public Node(T e, Node ptr){
        info = e;
        next = ptr;
    }
```

```
public void push (T e) {
        top = new Node(e, top);
}
```

```
public String toString() {
    if(isEmpty())
        return "top ---> null";

    String str = "top ---> ";
    Node tmp = top;

    while(tmp != null){
        str = str + tmp.info + " ---> ";
        tmp = tmp.next;
    }

    return str;
}
```

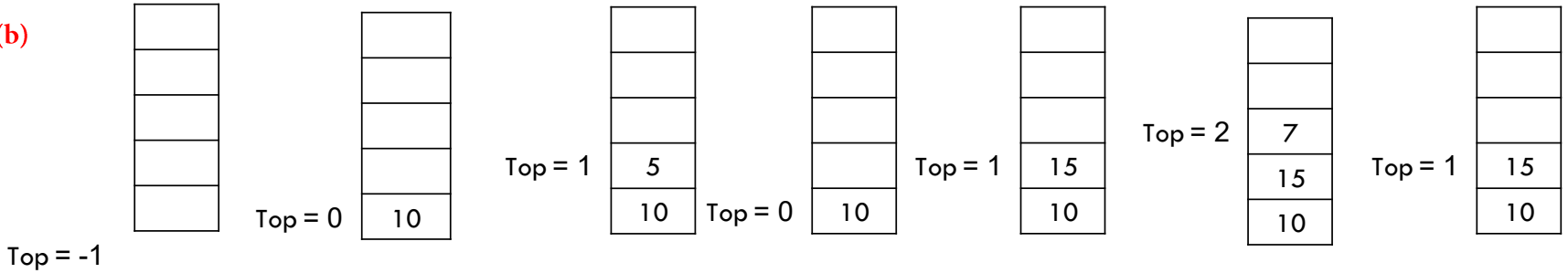
In the singly-linked list implementation, the top of the stack is the first node in the list.

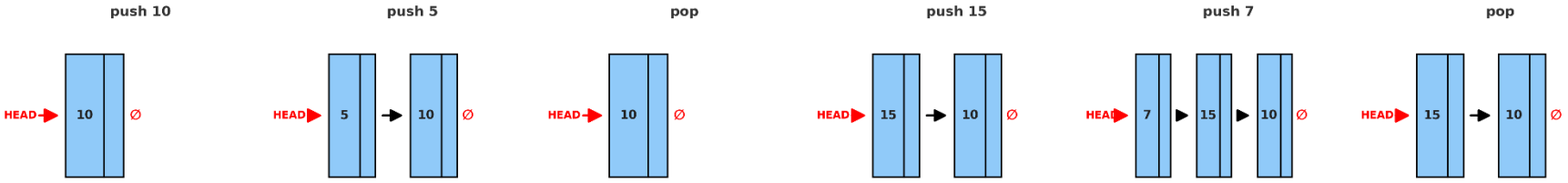
Stacks (continued)

(a)

```
push 10  →  push 5  →  pop  →  push 15  →  push 7  →  pop
```

(b)





(c)

(a) series of operations executed on an abstract stack

(b) the stack implemented with an array

(c) the stack implemented with a linked list

Application of stacks - Delimiter matching

- A delimiter is a character or symbol that indicates the beginning or end of a character string, word, or data item.

- Some types of delimiters are:
  - braces {}, brackets[], and parentheses(), angle brackets < > and /* */ delimiters.
  - Opening delimiters are: {, [, (, <, /* etc.
  - Closing delimiters are: },].), >, */ etc.

- We want to check if delimiters (like (), {}, []) are **balanced and properly nested**.
  - Balanced means:
    - Every **opening bracket** has a matching **closing bracket**
    - Brackets close in the **correct order**
  - Examples:
    - ✅ [(a+b) * {c-d}] ⟶ **Balanced**
    - ❌ [(a+b] * {c-d}) ⟶ **Imbalanced**

Brackets	Balanced
{ ( ) } [ ]	✓
{ (}) [ ]	✘
{ ( ) [ ] }	✓
{ }) ([ ]	✘

Application of stacks - Delimiter matching (cont'd)

- A simple algorithm that uses a stack to determine if the delimiters in an expression balanced:

1. Start with an empty stack.
2. Scan the string from left → right.
3. If the current character is an **opening delimiter** `(` `{` `[`
   → **push it onto the stack.**
4. If the current character is a **closing delimiter** `)` `}` `]`:
   - If the stack is **empty** → ❌ **Invalid** (unmatched closing delimiter).
   - **Pop** the top element from the stack.
   - If the popped opening delimiter **does not match** the closing delimiter
     → ❌ **Invalid** (mismatched delimiters).
5. *(Optional)* Ignore **non-delimiter characters.**
6. **After the scan:**
   - If the stack is **not empty** → ❌ **Invalid** (unclosed opening delimiter).
7. Otherwise → ✅ **Valid** (all delimiters are properly matched).

```
ForEach(character in string){
    if(character is opening delimiter)
        stack.push(character);
    else if (character is closing delimiter){
        if(stack.isEmpty())  return false;
        else{
            c =  stack.pop();
            if(c does not match  closing delimiter)
                return false;
        }
    }
}
return stack.isEmpty();
```

Opening bracket-> push

Closing bracket -> pop, compare, return FALSE if mismatch, otherwise continue.

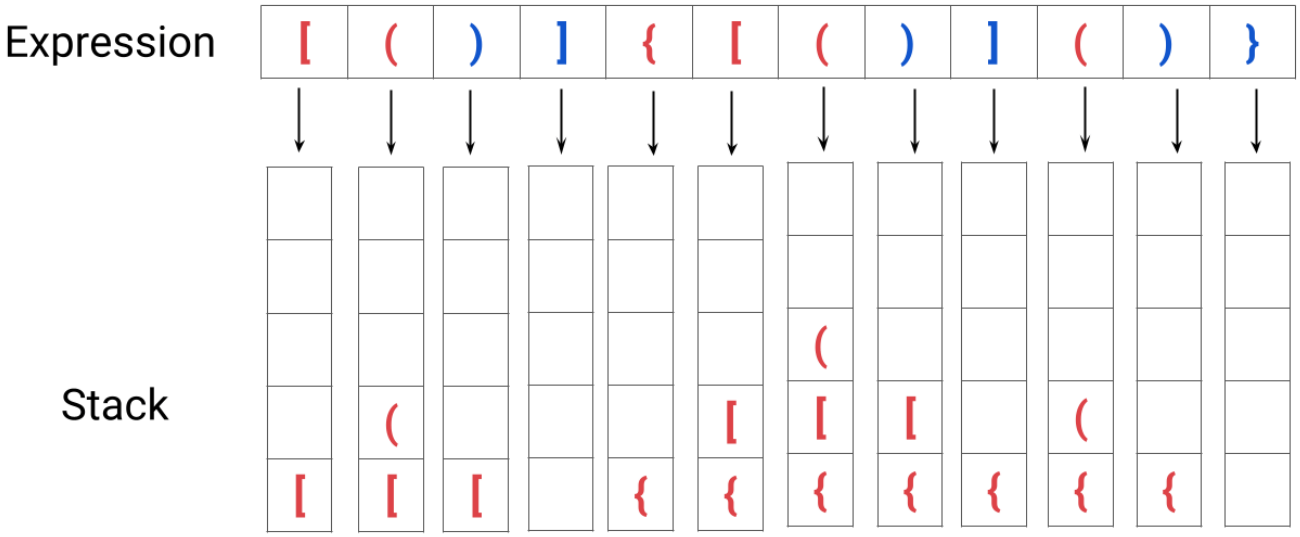
Return stack.isEmpty();

Time complexity: O(n)

Space complexity: O(n)

Delimiter matching: Example

- A simple algorithm that uses a stack to determine if the delimiters in an expression balanced:



Opening bracket-> push

Closing bracket -> pop, compare, return FALSE if mismatch, otherwise continue.

Return stack.isEmpty();

Application of Stacks: Evaluating Expressions

- Infix Expression – Operator is between operands (the usual human way). Example: A + B,
  - Needs operator precedence rules (or parentheses) to avoid ambiguity
  - Example: A + B * C ⟶ multiply happens before add.
- Postfix Expression – Operator comes after the operands. Example: AB +
  - No parentheses needed: order is unambiguous. Example: A B + C * (means (A + B) * C)
  - Easy for computers to evaluate using a stack:
- Prefix Expression – Operator comes before the operands. Example: +AB
  - No parentheses needed: order is unambiguous. Example: * + ABC (means (A + B) * C)
  - Easy for computers to evaluate using a stack:

Notation	Syntax	Example	Meaning	Needs Parentheses?
Infix	Operand **Operator** Operand	`A + B * C`	`A + (B * C)`	✅ Yes (or rules)
Postfix	Operand Operand **Operator**	`A B + C *`	`(A + B) * C`	❌ No
Prefix	**Operator** Operand Operand	`* + A B C`	`(A + B) * C`	❌ No

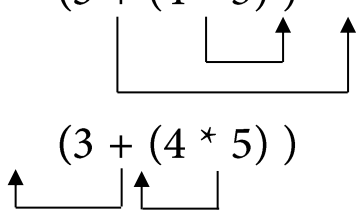
Infix to Postfix/Prefix conversion (manual)

- Completely parenthesize the infix expression according to the order of priority you want.
- Move each operator to its corresponding right/left parenthesis for postfix /prefix respectively.
- Remove all parentheses.
- Example: convert 3 + 4 * 5 to postfix and prefix
  - Infix to postfix

3 + 4 * 5 (3 + (4 * 5)) 3 4 5 * +

  - Infix to prefix

3 + 4 * 5 (3 + (4 * 5)) + 3 * 4 5



Infix Notation	Postfix Notation	Prefix Notation
16 / 2	16 2 /	/ 16 2
(2 + 14)* 5	2 14 + 5 *	* + 2 14 5
2 + 14 * 5	2 14 5 * +	+ 2 * 14 5
(6 – 2) * (5 + 4)	6 2 - 5 4 + *	* - 6 2 + 5 4

Infix to Postfix conversion(Using Stack)

1. Start with an **empty stack** and **empty postfix string**.
2. Scan the infix expression left → right.
3. **If operand:** add to postfix.
4. **If** '(': push onto stack.
5. **If** ')': pop from stack to postfix until '(' is found; discard '('.
6. **If operator:**
   - Pop from stack to postfix while top has **higher/equal precedence**.
   - Push current operator onto stack.
7. **After scanning:** pop all remaining operators to postfix.

Infix: (A + B * C) * D - E

Step	Symbol	Stack	Postfix	Action
1	(	(		Push '(' onto stack
2	A	(	A	Operand → add to postfix
3	+	(+	A	Operator → push onto stack
4	B	(+	AB	Operand → add to postfix
5	*	(+ *	AB	Operator → push onto stack (higher precedence than '+')
6	C	(+ *	ABC	Operand → add to postfix
7	)		ABC*+	Pop until '(' → add operators to postfix ('*' then '+'), discard '('
8	*	*	ABC*+	Operator → push onto stack
9	D	*	ABC*+D	Operand → add to postfix
10	-	-	ABC*+D*	Pop '*' (higher precedence), push '-'
11	E	-	ABC*+D*E	Operand → add to postfix
12	End		ABC*+D*E-	Pop remaining operator '-' → add to postfix

Postfix: ABC*+D*E-

Application of Stacks - Evaluating Postfix Expression (Cont'd)

- The following algorithm uses a stack to evaluate a postfix expression.

```
Start with an empty stack
   for (each Token in the expression) {
       if (the Token is an operand)
           Push the operand onto the stack
       else if (the Token is an operator){
           Pop operand1 from the stack
           Pop operand2 from the stack
               result = operand2 operator operand1
           Push the result onto the stack
                   }
   }
   pop the only operand from the stack:  this is the result of the evaluation
```

Read a token

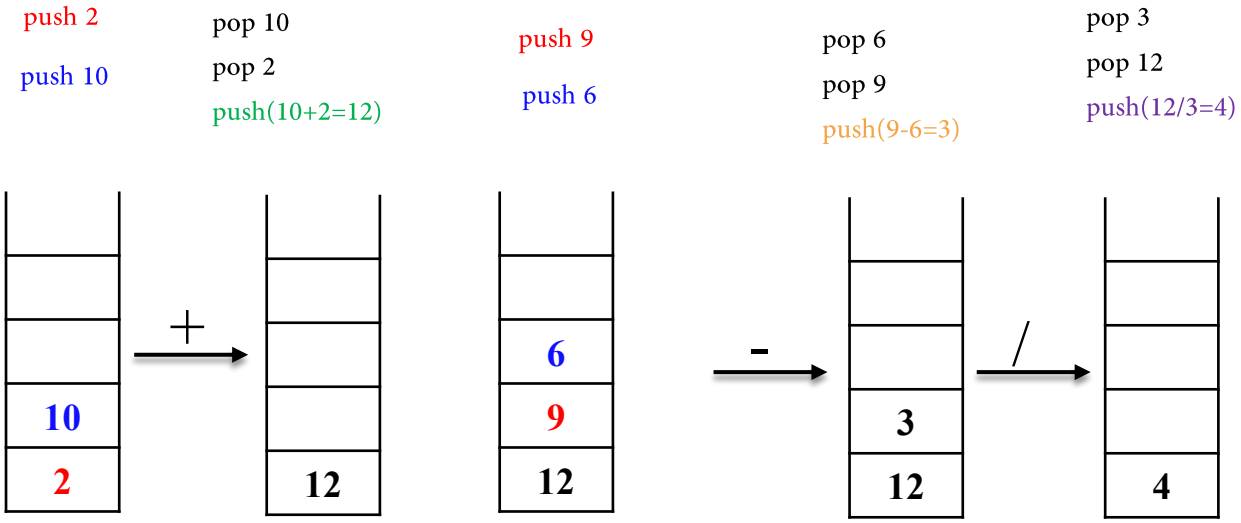
If number -> push to stack.

If an operator -> pop TWO numbers, apply the operator on the numbers, and push the result.

Evaluating Postfix Expression (Cont'd)

- Example: Consider the postfix expression, **2 10 + 9 6 - /**, which is **(2 + 10) / (9 - 6)** in infix, the result of which is 12 / 3 = 4.
  - The following is a trace of the postfix evaluation algorithm for the postfix expression:

token	Stack	Operation
2	2	push(2)
10	2 10	push(10)
+	12	pop 10 , pop 2 2 +10 =12 , push(12)
9	12 9	push(9)
6	12 9 6	push(6)
-	12 3	pop 6 , pop 9 9 – 6 = 3, push(3)
/	4	pop 3, pop 12 12 / 3 = 4, push(4)



Queues

- What is a Queue?

- Queue Implementations:
  - Queue As Linked List
  - Queue As Array
  - Queue As Circular Array

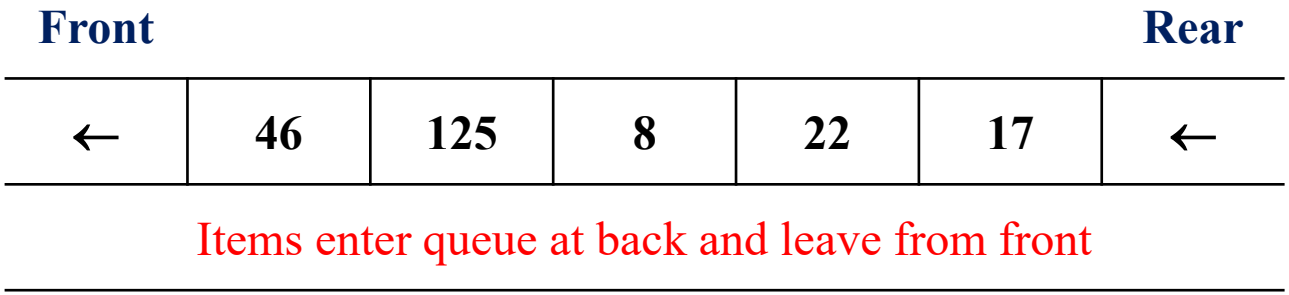
- Priority Queues

Queue

- State:
  - List or array of ordered items
- Behavior
  - enqueue(item)
  - dequeue()
  - peek()
  - isEmpty()

Queues

- Queues are linear data structures that follows the **FIFO** (First In, First Out) principle, in which we add elements to one end and remove them from the other end.
  - A queue is a waiting line that grows by adding elements to its end and shrinks by taking elements from its front



  - The first item to be inserted (en-queued) is the first to be deleted (de-queued).
  - A queue is therefore called a First In First Out (FIFO) data structure.
- Queue operations:
  - enqueue: insert an element at the rear of the queue
  - dequeue: delete an element at the front of the queue
  - peek: get the element at the front without deleting it

QueueAsLinkedList Implementation

```java
public class QueueAsList<T> {

    private class Node {
        T info;
        Node next;
        public Node(T e){
            info = e;
            next = null;
        }
    }

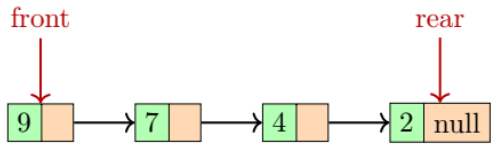
    private Node front, rear;

    public QueueAsList(){
        front = rear = null;
    }

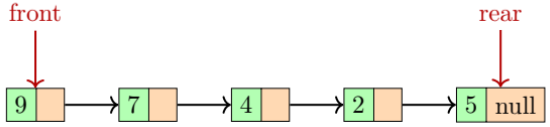
    public boolean isEmpty(){
        return (front == null);
    }

    public T peek() {
        if (isEmpty())
            return null;
        return (T) front.info;
    }
```

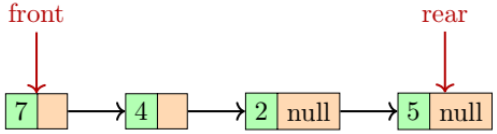
```java
public void enqueue (T e){
    Node node = new Node(e);
    if (isEmpty())
        front = node;
    else
        rear.next =  node;
    rear = node;
}
public T dequeue() {
    if (isEmpty())
        return null;
    T result = (T) front.info;
    front = front.next;
    if (isEmpty())
        rear = null;
    return result;
}
```



Queue after inserting a node

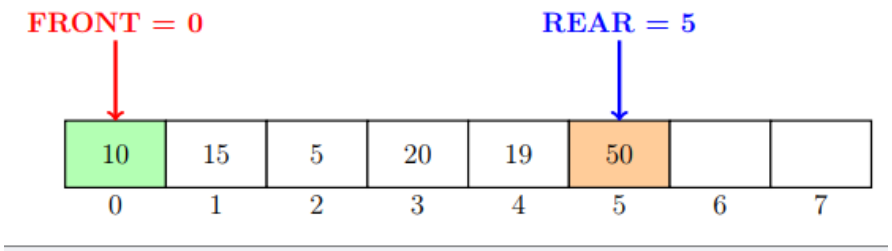


Queue after deleting a node

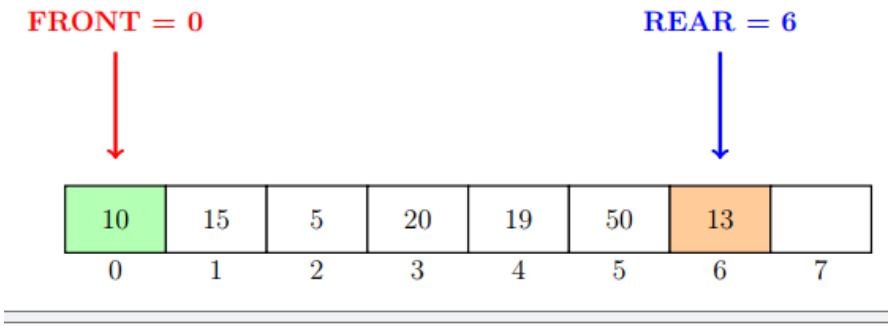


QueueAsArray Implementation

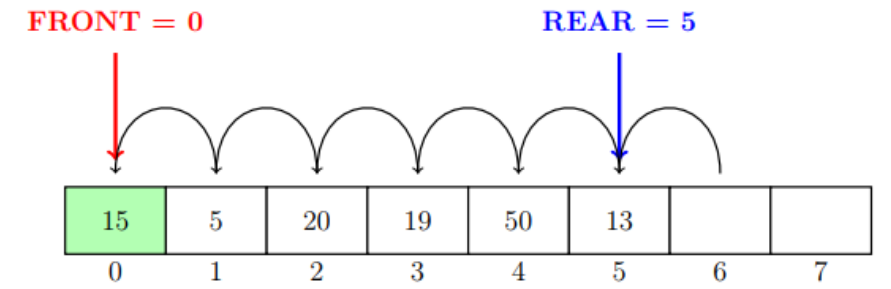
- In the QueueAsArray implementation that follows:
  - front == -1 and rear == -1 if the queue is empty
  - The front of the queue is always array[0]
  - If the queue is not empty, the rear of the queue is array[count – 1] where count is the number of elements in the queue.
  - The queue is full if rear +1 == capacity where capacity is the maximum array size
  - Because of shifting elements, the dequeue operation is O(n)



INSERT at the END (enqueue 60)



DELETE (dequeue 10)



QueueAsArray

```java
public class QueueAsArray<T> {

    private int front;
    private int rear;
    private final int capacity;
    private final T[] queue;
    public QueueAsArray(int capacity) {
        front = rear = -1;
        this.capacity = capacity;
        queue = (T[]) new Object[capacity];
    }

    public boolean isEmpty(){
        return front == -1;
    }

    public boolean isFull(){
        return rear+1 == capacity;
    }

    public  T peek() {
        if (isEmpty())
            return null;
        return queue[front];
    }
```

```java
public boolean enqueue(T data)  {
    if (isFull())
        return false;
    if(isEmpty())
        front++;
    rear++;
    queue[rear] = data;
    return true;
}

public T dequeue()  {
    if (isEmpty())
        return  null;
    T data = queue[front];
    if (rear == 0) {//  one element
        rear = front = -1;
    }
    else { // more the one element
        for (int i = 0; i <= rear - 1; i++) {
            queue[i] = queue[i + 1];
        }
        rear--;
    }
    return data;
}
```

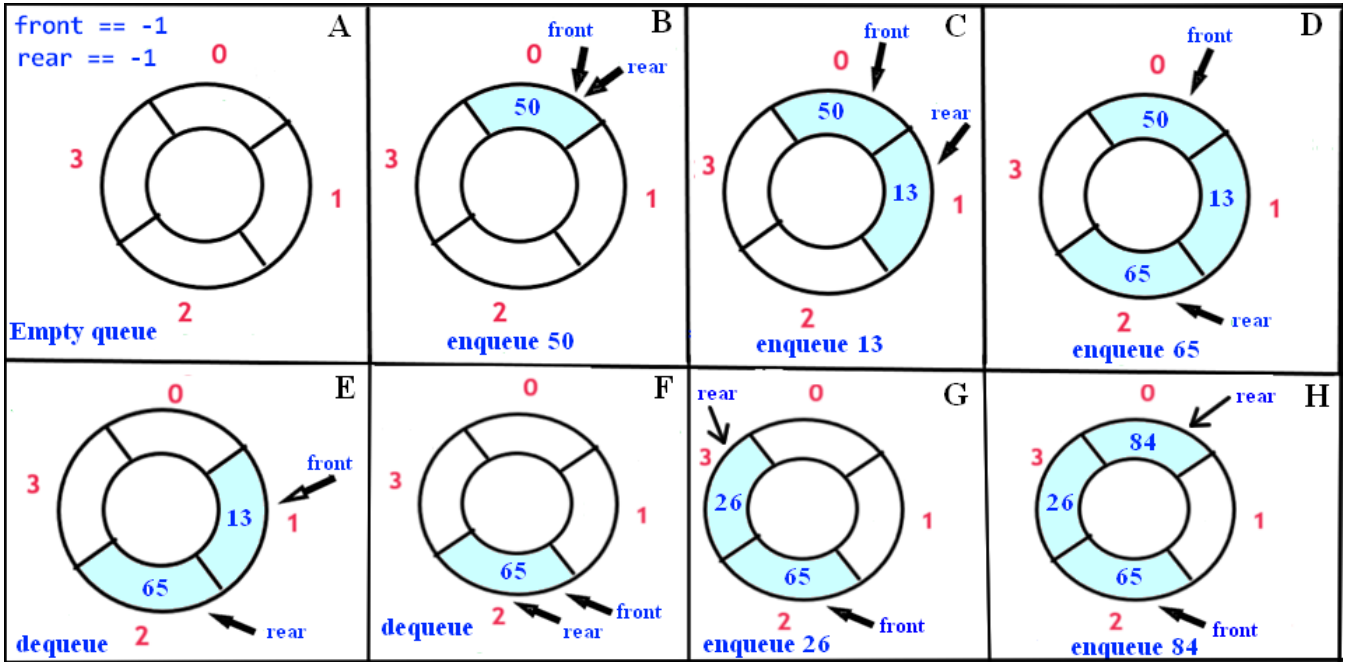
O(1)

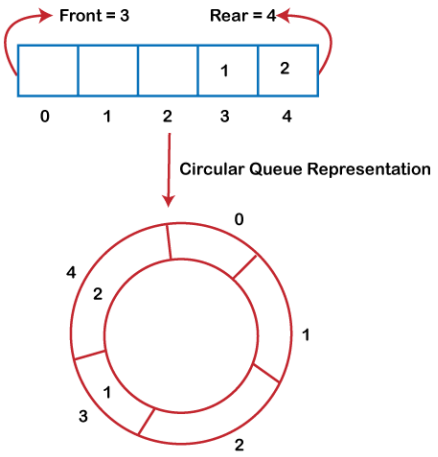
O(n)

```java
public String toString() {
    if (isEmpty())
        return "";
    String output = "[ ";
    for (int i = 0; i <= rear; i++) {
        output = output + queue[i] + "   ";
    }
    return output + "]";
}
```

QueueAsCircularArray Implementation

- A **circular queue** is a **linear queue** in which the **last position is connected back to the first position** to make a circle.
- By using modulo arithmetic for computing array indexes, we can have a queue implementation in which each of the operations enqueue, dequeue, and peek has complexity O(1)





QueueAsCircularArray…

```java
public class QueueAsCircularArray <T>{

    private T[] queue;
    private int front, rear, capacity;

    public QueueAsCircularArray(int capacity) {
        this.capacity = capacity;
        queue = (T[])new Object[capacity];
        front = rear = -1;
    }

    public boolean isEmpty() {
        return  front == -1;
    }

    public boolean isFull() {
        return (rear + 1) % capacity == front;
    }

    public T peek() {
        if(isEmpty())
            return null;
        else
            return queue[front];
    }
```

```java
public boolean enqueue(T data) {
    if(isFull()) {
        return false;
    }
    if(isEmpty()){
        front = rear = 0;
    } else {
        rear = (rear + 1) % capacity;
    }
    queue[rear] = data;

    return true;
}
```

O(1)

```java
public T dequeue() {
    if(isEmpty())
        return null;
    T data = queue[front];
    if(front == rear)
        front = rear = -1;
    else
        front = (front + 1) % capacity;

    return data;
}
```

O(1)

```java
public String toString() {
    if(isEmpty())
        return "[]";
    String output = "[";
    int index = front;
    do   {
        output+= queue[index] + ", ";
        index = (index + 1) % capacity;
    } while (index != (rear + 1) % capacity);

    return output + "]";
}
```

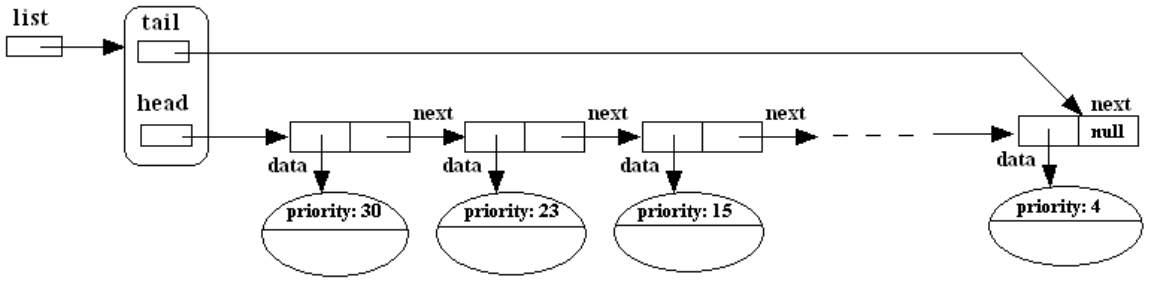
Priority Queues

- In a normal queue,
  - the enqueue operation adds an item at the end of the queue,
  - the dequeue operation removes an item from the front of the queue.
- A priority queue is a collection of elements such that
  - Each element has a priority value and such that the order in which elements are deleted and processed comes from the following rules:
  - An element of higher priority is processed (removed) before any element of lower priority.
  - Two elements with same priority are processed according to the order in which they were added to the queue.
- The elements in a priority queue can have the following structure:

```
private class Node {
int priority;
int data;
Node next;
}
```

Priority Queue Implementation

- One implementation of Priority Queue uses a singly-linked list that has head and tail references:



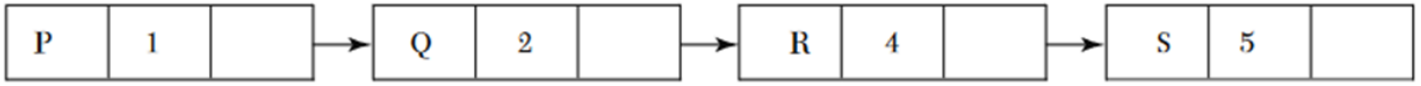
- Default: All elements are sorted according to their priorities
  - enqueue is O(n) – insert in the right position.
  - dequeue is O(1) – it is always at the beginning

- All elements are stored according to their entry ordered
  - enqueue is O(1) – just append the new element.
  - dequeue is O(n) - linear scan to find the element with the highest priority.

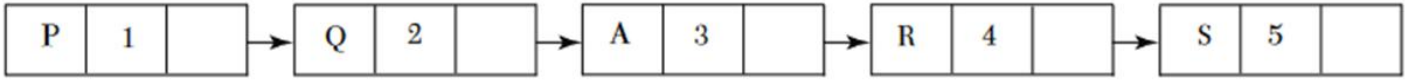
- In a later lesson we will study another implementation of priority queue that uses a data structure called the binary-heap

All elements are sorted according to their priorities

- Insertion:
  - Traverse the entire queue until we find a node which has a lower priority than the new element.



Linked priority queue before insertion.

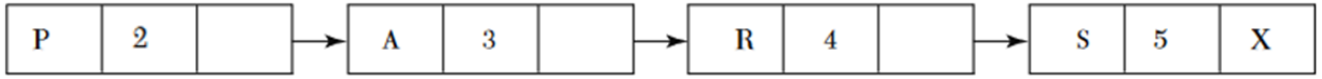


Linked priority queue after inserting a new element.

- Deletion:
  - The first node from the priority queue is deleted



Linked priority queue before deletion.



Linked priority queue after deleting the first node.

Applications of Stacks & Queues

- Queues are commonly used in systems where the waiting line must be maintained
- A Priority Queue can be used in any application that uses a set of elements of various priorities where the element of highest priority needs to be accessed first.
- A stack is used in systems where operations must follow a last-in, first-out order

Stacks	Queue	Priority Queue
Expression Processing	CPU Scheduling	CPU Scheduling
Function Call Management	Access to shared resources	
Backtracking Algorithms	Network and Communication	
Undo/Redo with Two Stacks	Buffering audio/video frames	
	Simulation: Queueing theory	
Auxiliary data structure for some algorithms: Tree traversal	Auxiliary data structure for some algorithms: Tree traversal	Auxiliary data structure for some algorithms: Huffman Codes Dijkstra's Algorithm for Shortest Paths Prim's Algorithm

Complexity

Operation	Stack as array	Stack as linked list with operations on head	Stack as linked list with operations on tail
push	O(1)	O(1)	O(1)
pop	O(1)	O(1)	O(n)

Operation	Queue as array	Queue as linked list with (front and rear)	Queue as linked list with front only	Circular Queue (array)
enqueue	O(1)	O(1)	O(n)	O(1)
dequeue	O(n)	O(1)	O(1)	O(1)